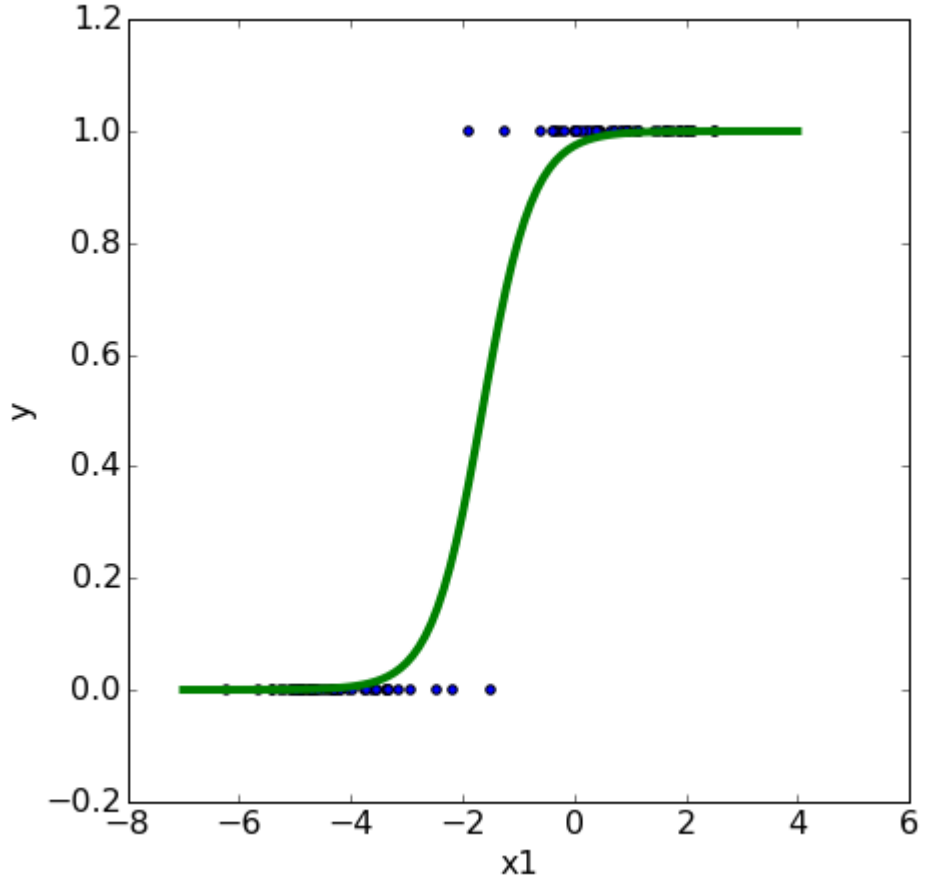
Logistic regression forms the basis of complex neural networks

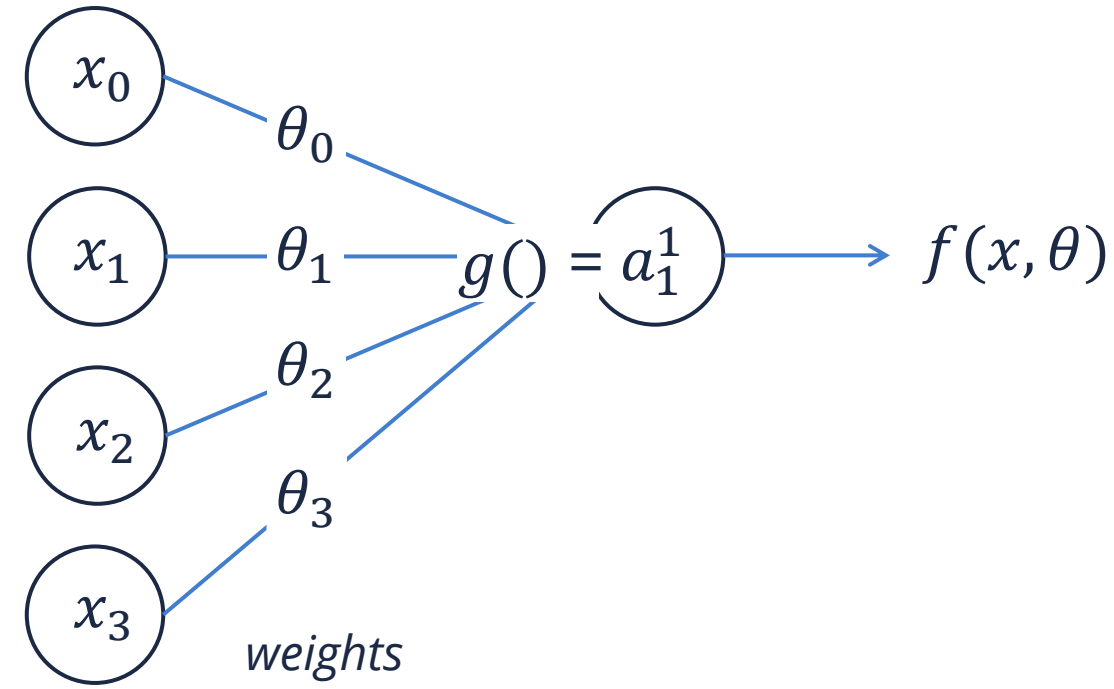


$$f(x,\theta) = g(\theta_0 + \theta_1 x_1)$$

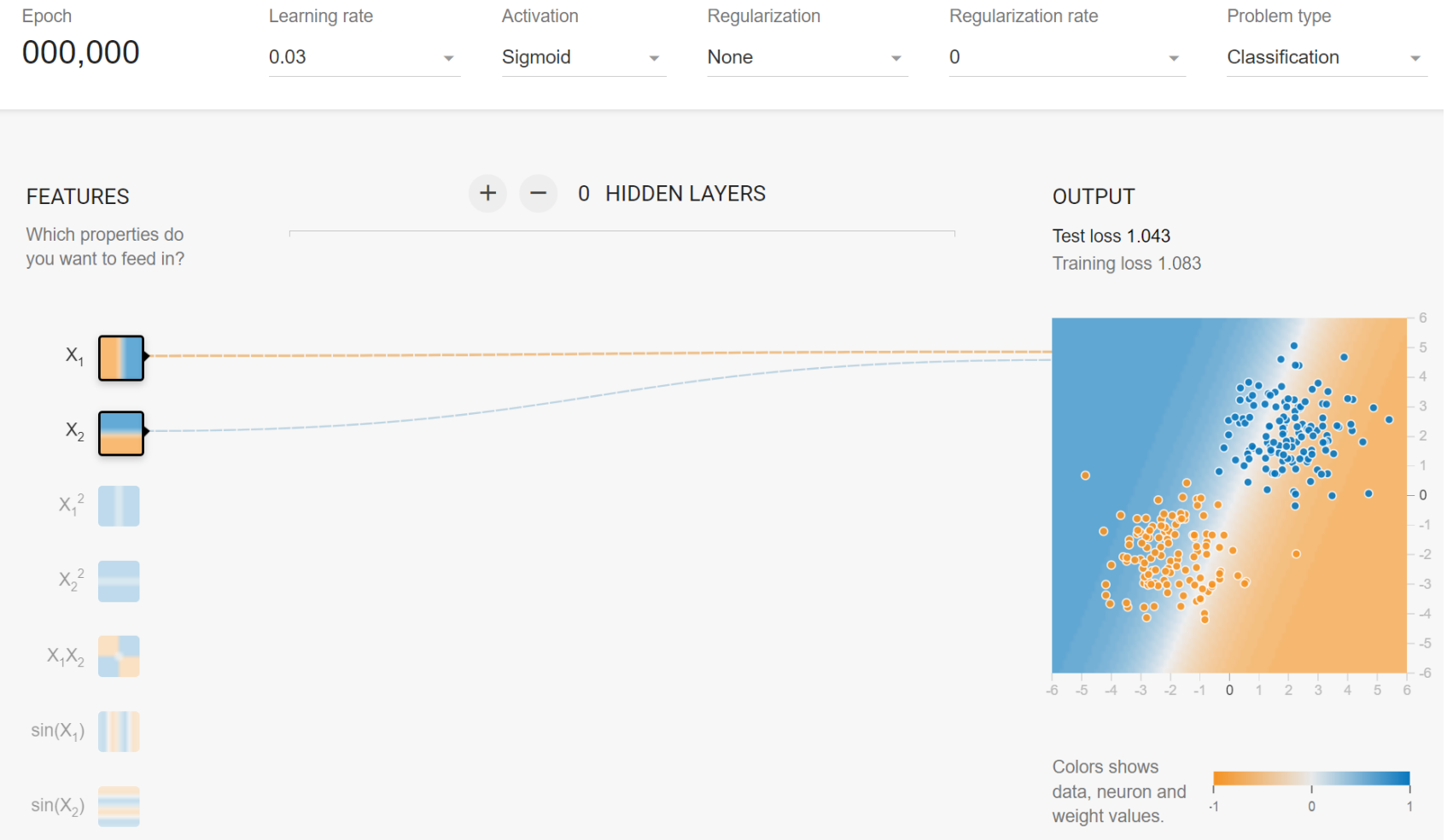
More inputs,
More parameters

$$f(x,\theta) = g(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_m x_m)$$

Logistic regression could be visualized as a network of information, flowing from features and weights to the output

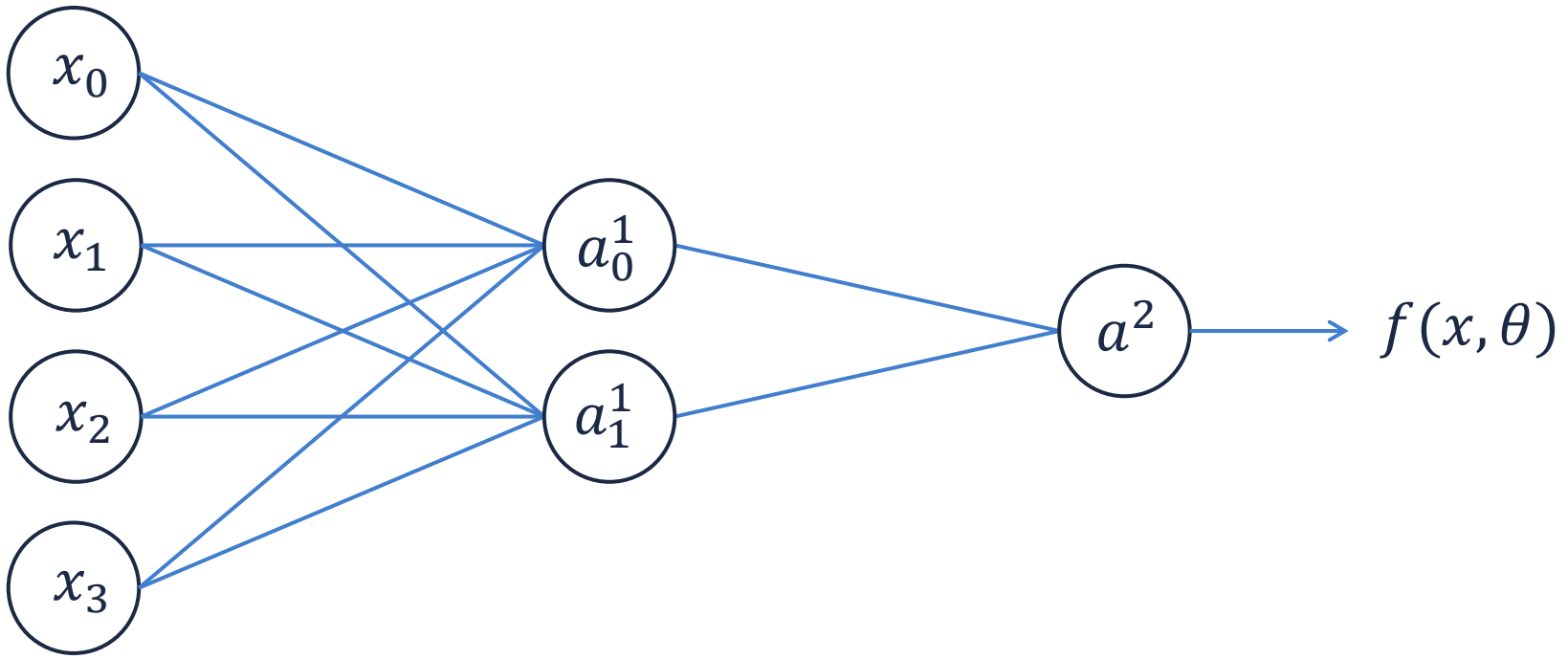


Logistic regression could be visualized as a network of information, flowing from features and weights to the output



A neural network is simply a layered stack of regression models

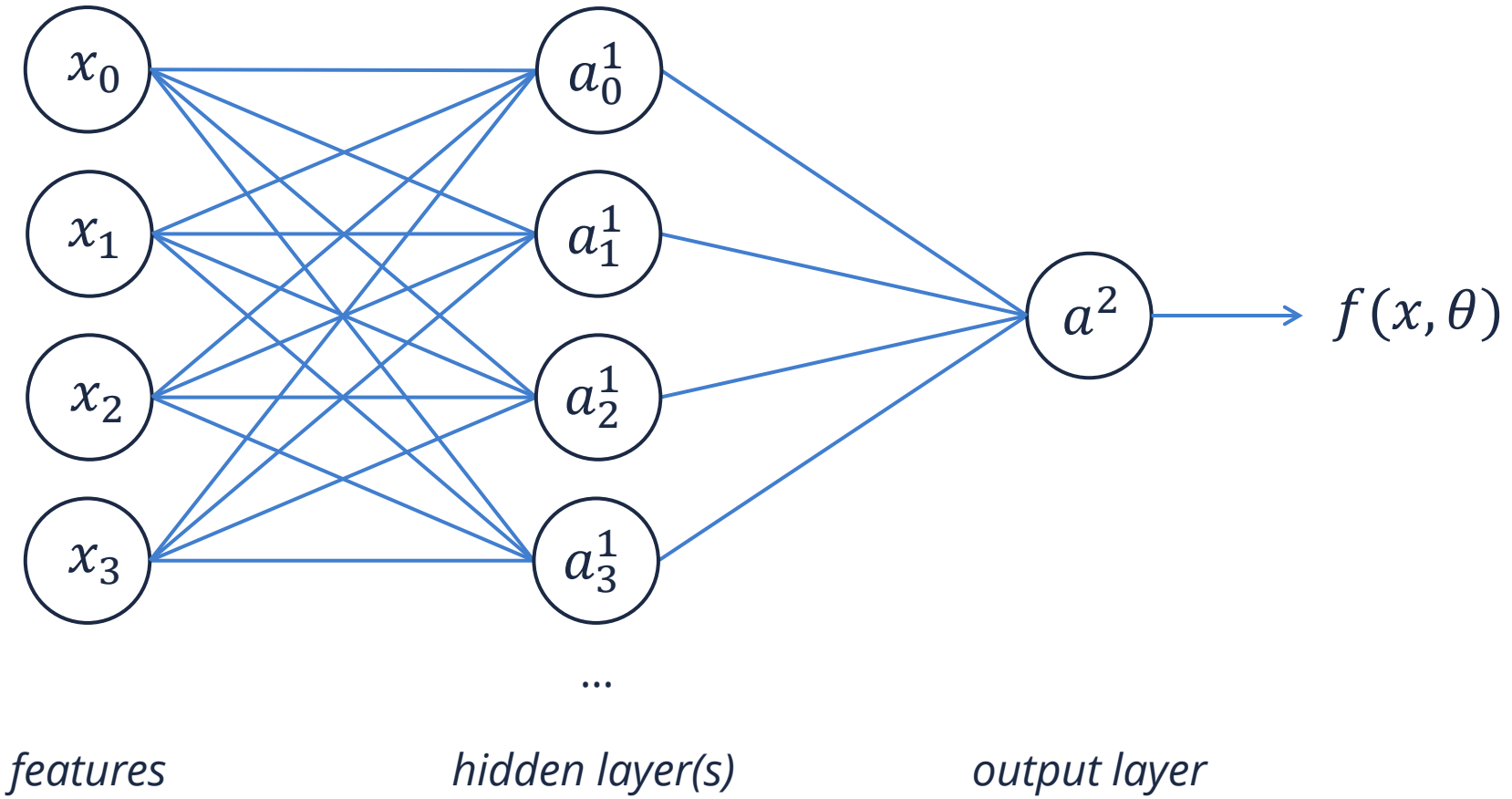
The features for each node (a regression model) are the output from the nodes in the previous layer



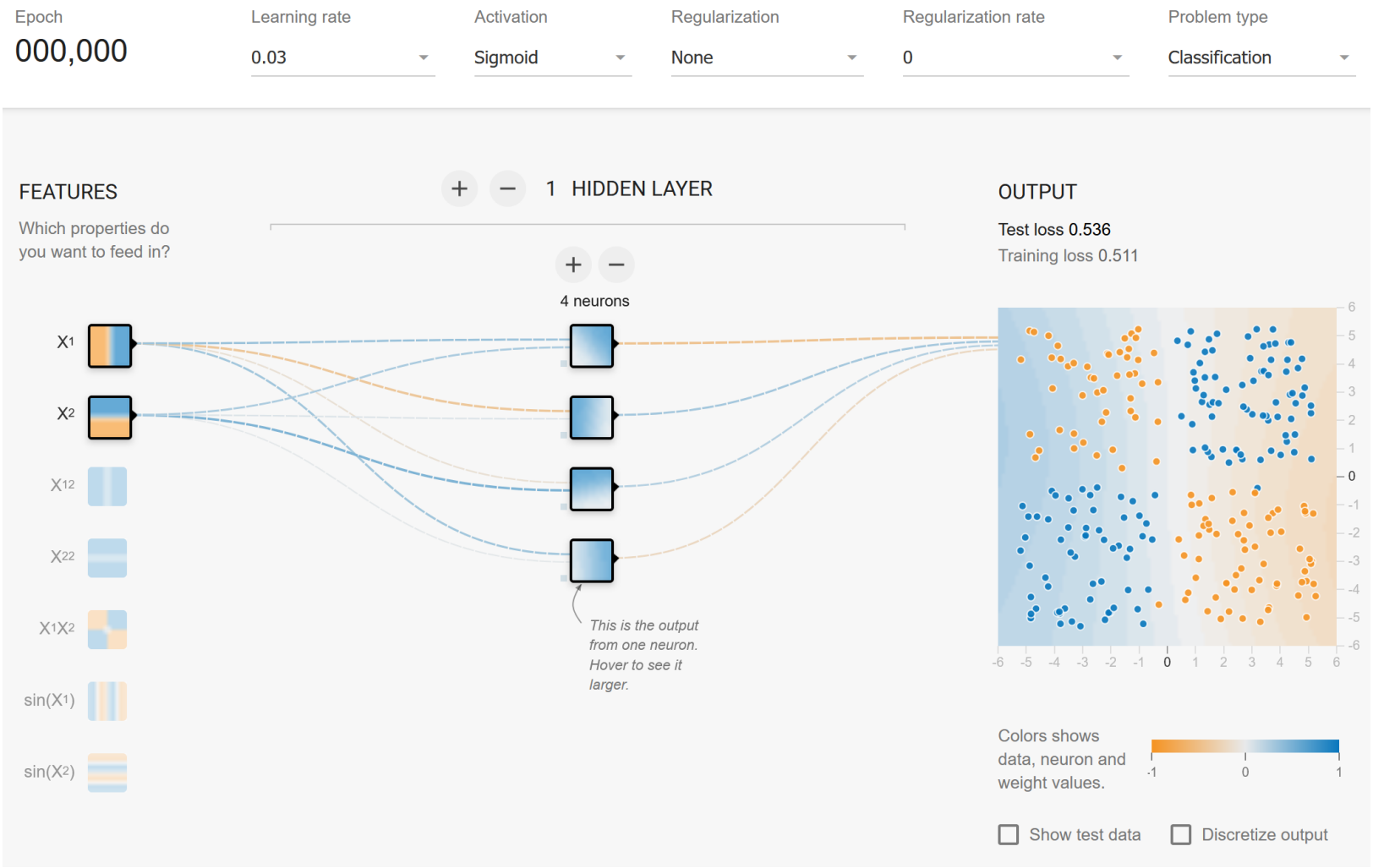
features *hidden layer(s)* *output layer*

A neural network is simply a layered stack of regression models

The features for each node (a regression model) are the output from the nodes in the previous layer

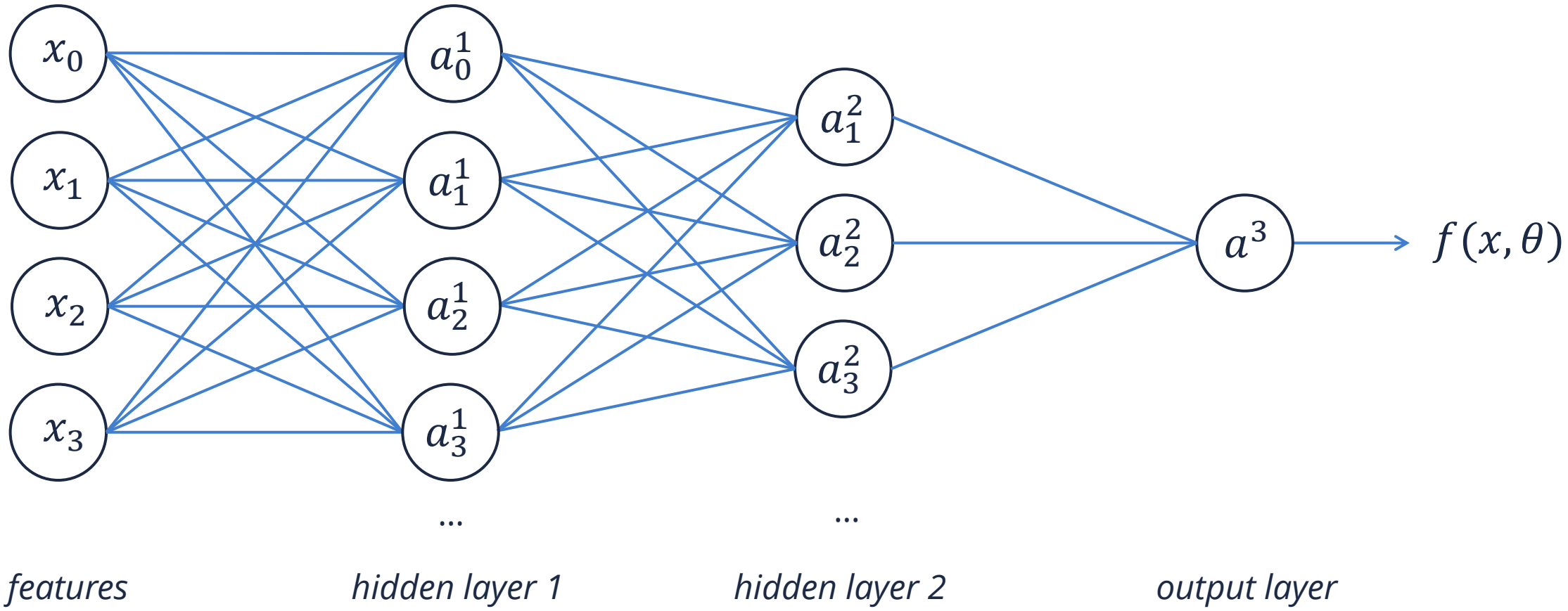


A neural network is simply layered stacks of logistic regressions

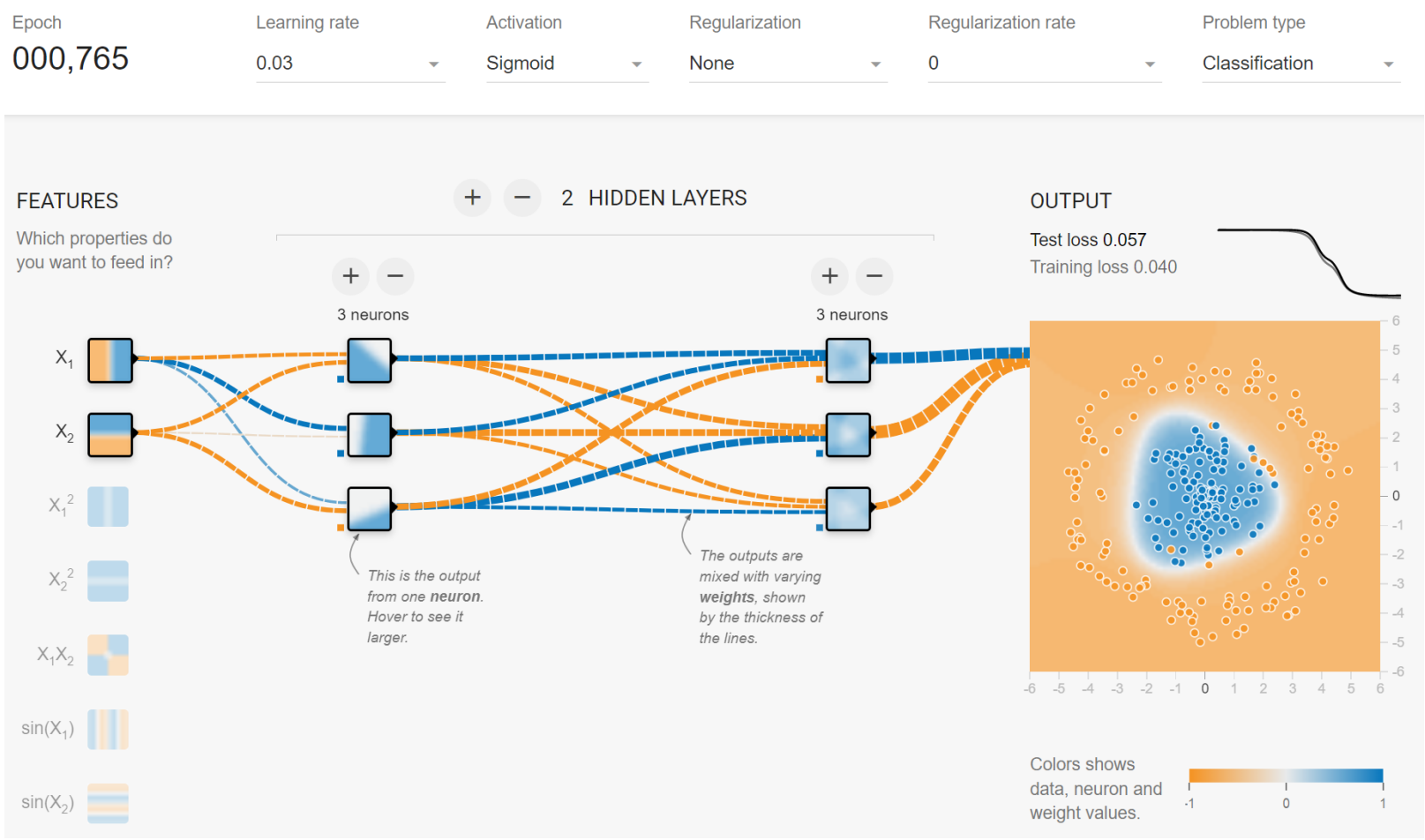


https://playground.tensorflow.org

A deep neural network adds many layers to capture complex patterns



A deep neural network adds many layers to capture complex patterns



https://playground.tensorflow.org

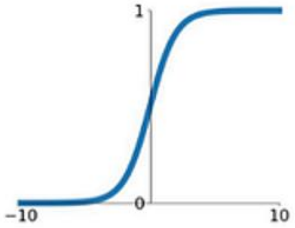
Activation functions can be added on the output of each node

➡️ Capture non-linear relationships

➡️ Avoid exploding or diminishing outputs in deep networks

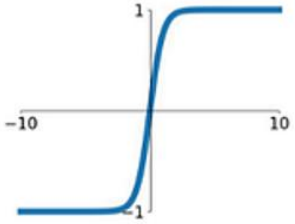
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



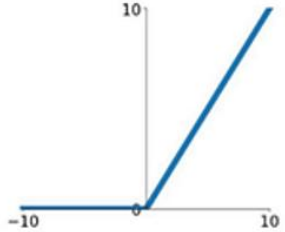
tanh

$$\tanh(x)$$



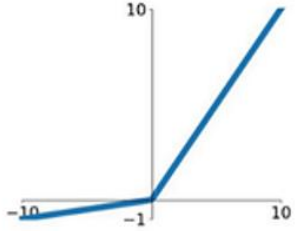
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

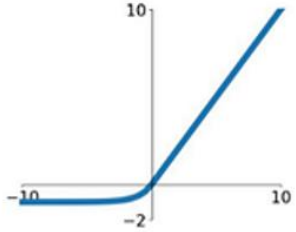


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

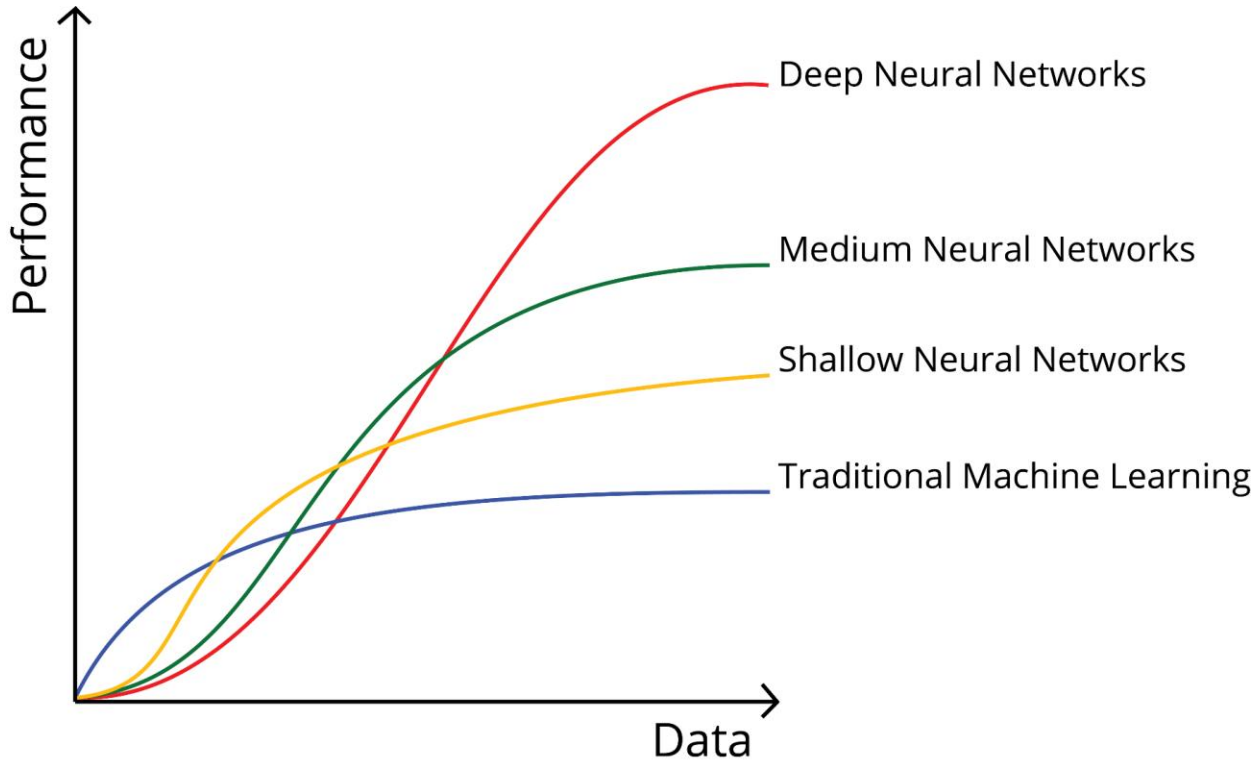
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Do we need to go deeper?

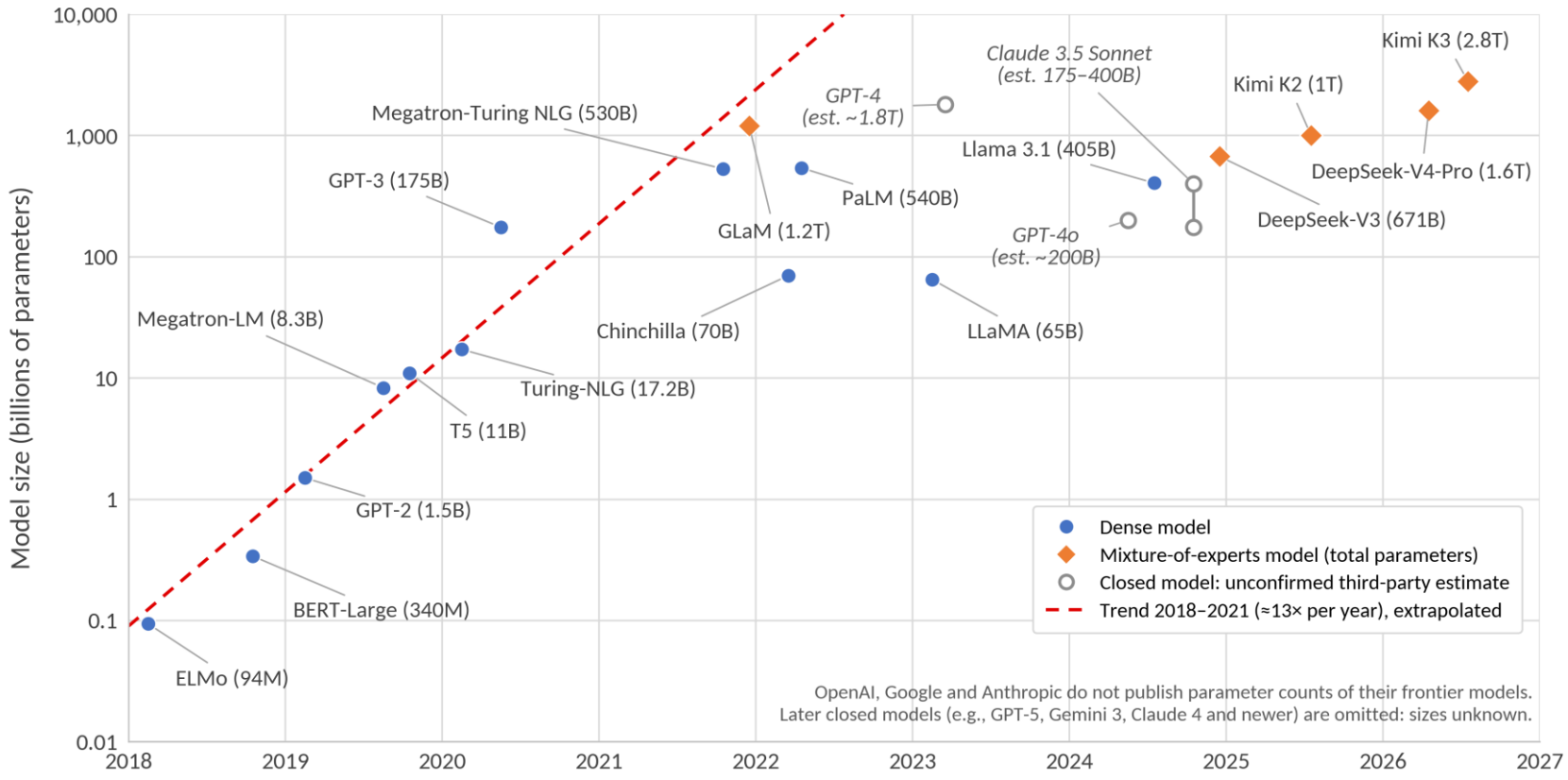
Deeper networks…

- ➡️ **… can learn more complex patterns**
- ➡️ **… require more data to be trained**



For complex tasks, deeper seems better…

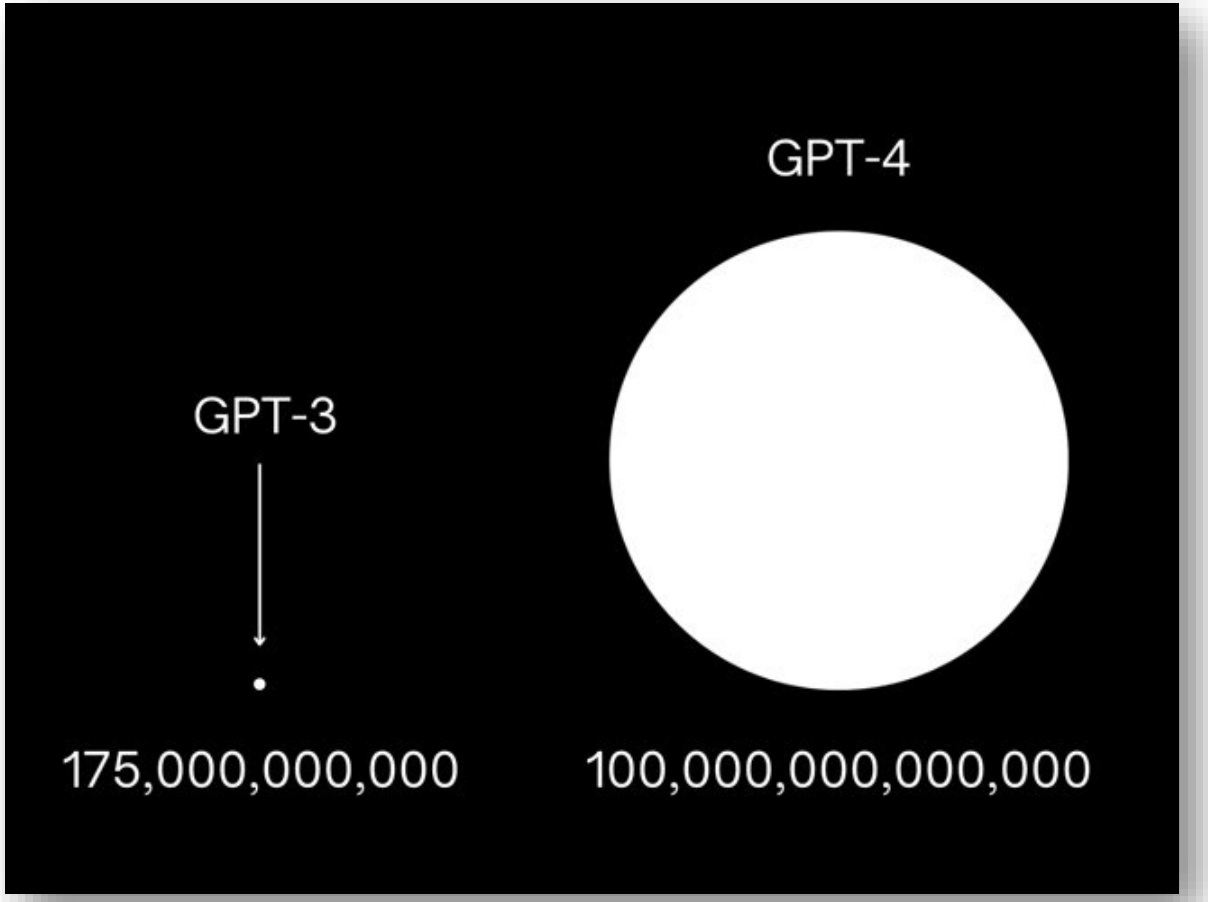
However, more complicated architecture setups are required to make the models learn well.



Sources: original publications and model cards (month of first public release); 2018–2021 selection as in the Microsoft/NVIDIA MT-NLG blog post (2021).
Closed-model estimates: GPT-4 from SemiAnalysis (2023), NVIDIA (2024) and Microsoft (2024); GPT-4o and Claude 3.5 Sonnet from Epoch AI (2024) and Microsoft (2024).

For complex tasks, deeper seems better…

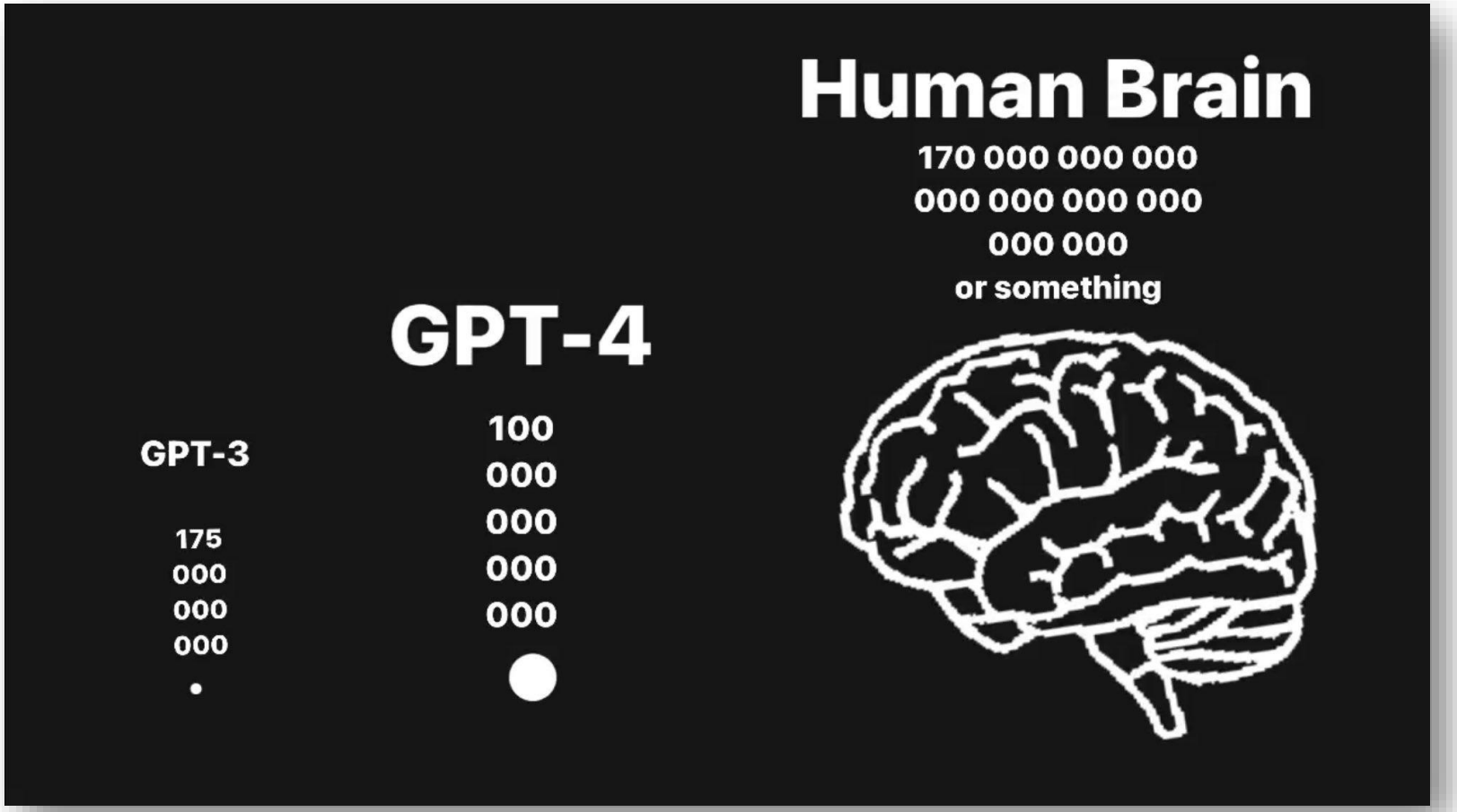
However, more complicated architecture setups are required to make the models learn well.



https://openai.com/product/gpt-4

For complex tasks, deeper seems better...

However, more complicated architecture setups are required to make the models learn well.



https://openai.com/product/gpt-4

The *loss function* should be minimized to find the optimal model

regression ➡️ MSE

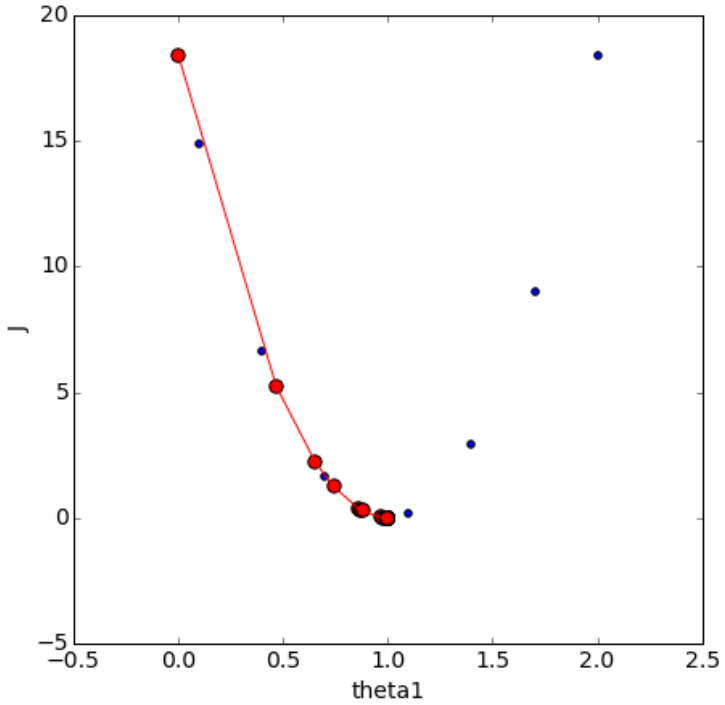
binary classification ➡️ binary cross entropy

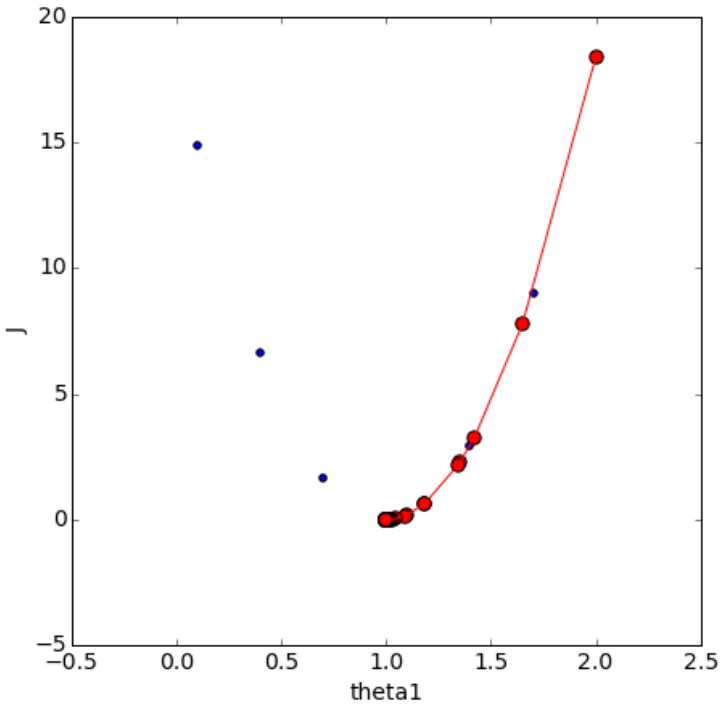
multi-class classification ➡️ categorical cross entropy

… ➡️ …

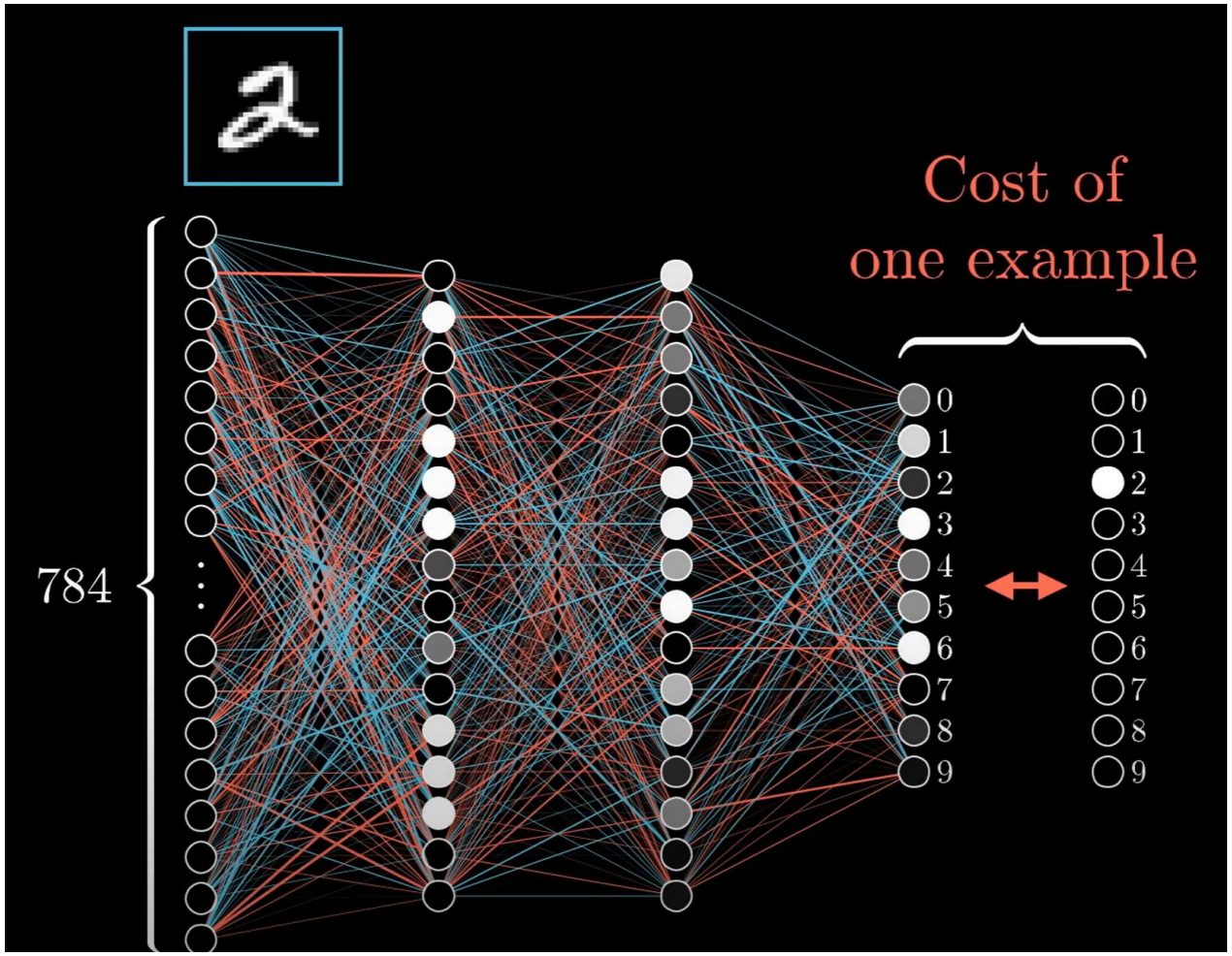
Neural nets also rely on *gradient descent* to minimize the loss

Iteratively adjusting the weights and biases in the direction that reduces the loss



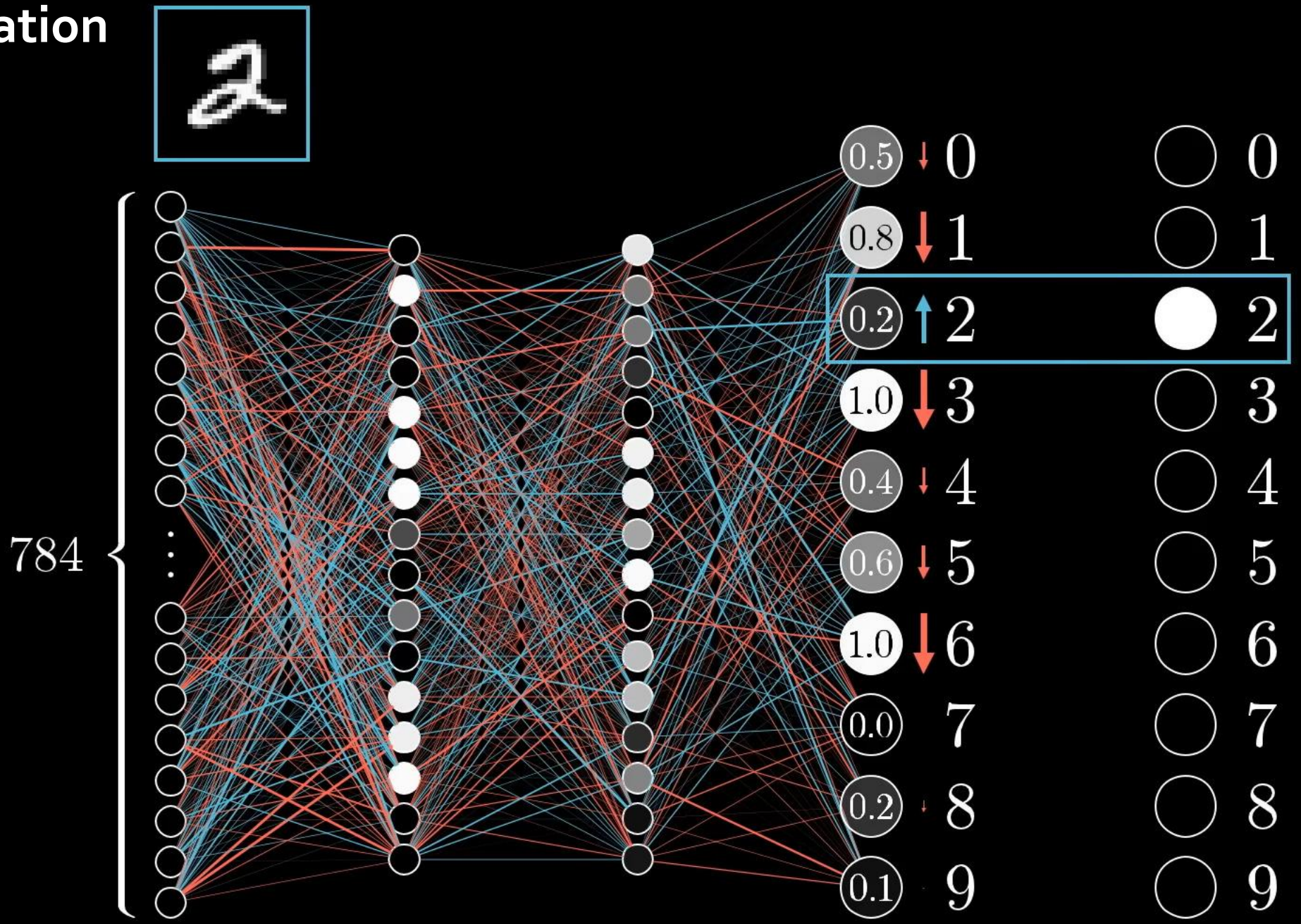


The big trick is to use *backpropagation* to efficiently calculate gradients for each parameter across many layers



https://www.youtube.com/watch?v=Ilg3gGewQ5U&t=548s

backpropagation



https://youtu.be/Ilg3gGewQ5U?si=hvb1oqZ-GgHn3kl9&t=409

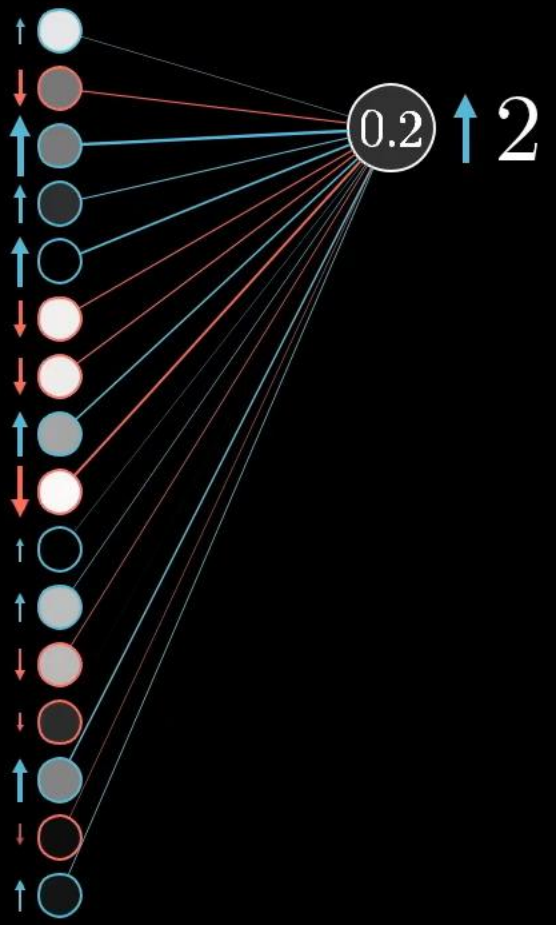


$$0.2 = \sigma(w_0 a_0 + w_1 a_1 + \cdots + w_{n-1} a_{n-1} + b)$$

Increase b

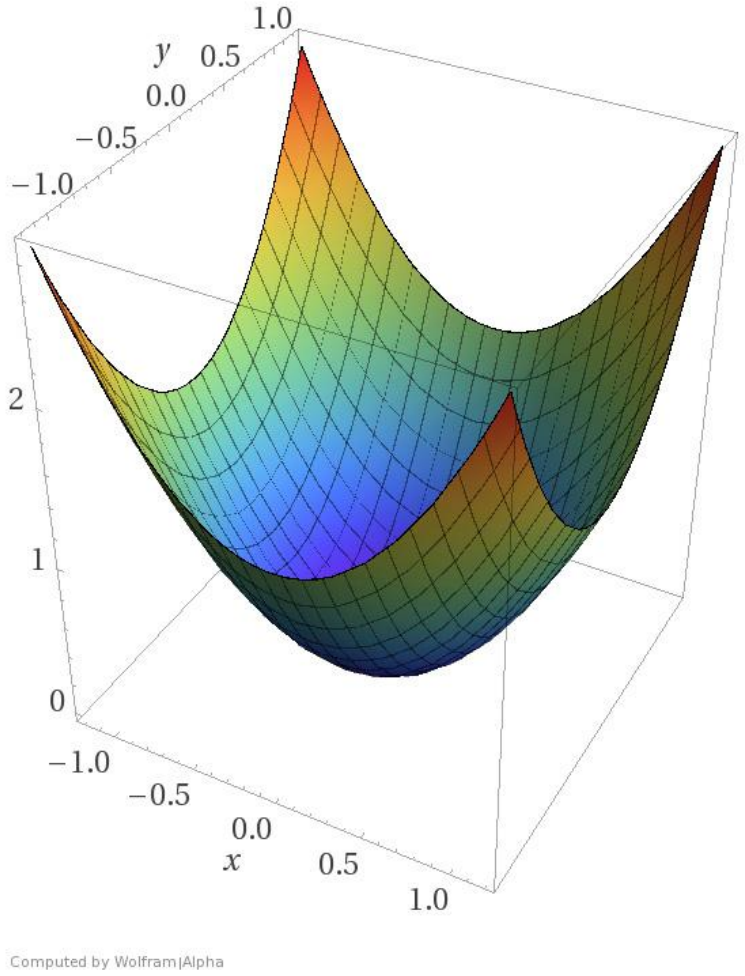
Increase w_i
in proportion to a_i

Change a_i

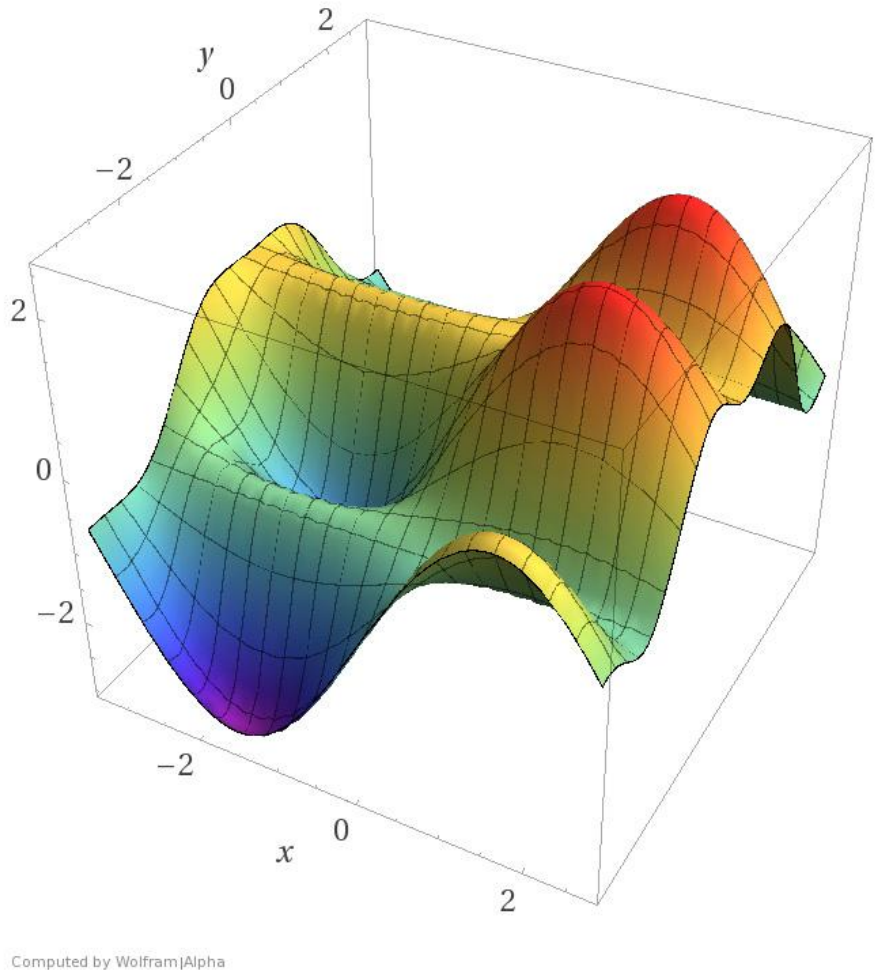


https://youtu.be/Ilg3gGewQ5U?si=hvb1oqZ-GgHn3kl9&t=409

The loss surface can be highly complex, with many local minima

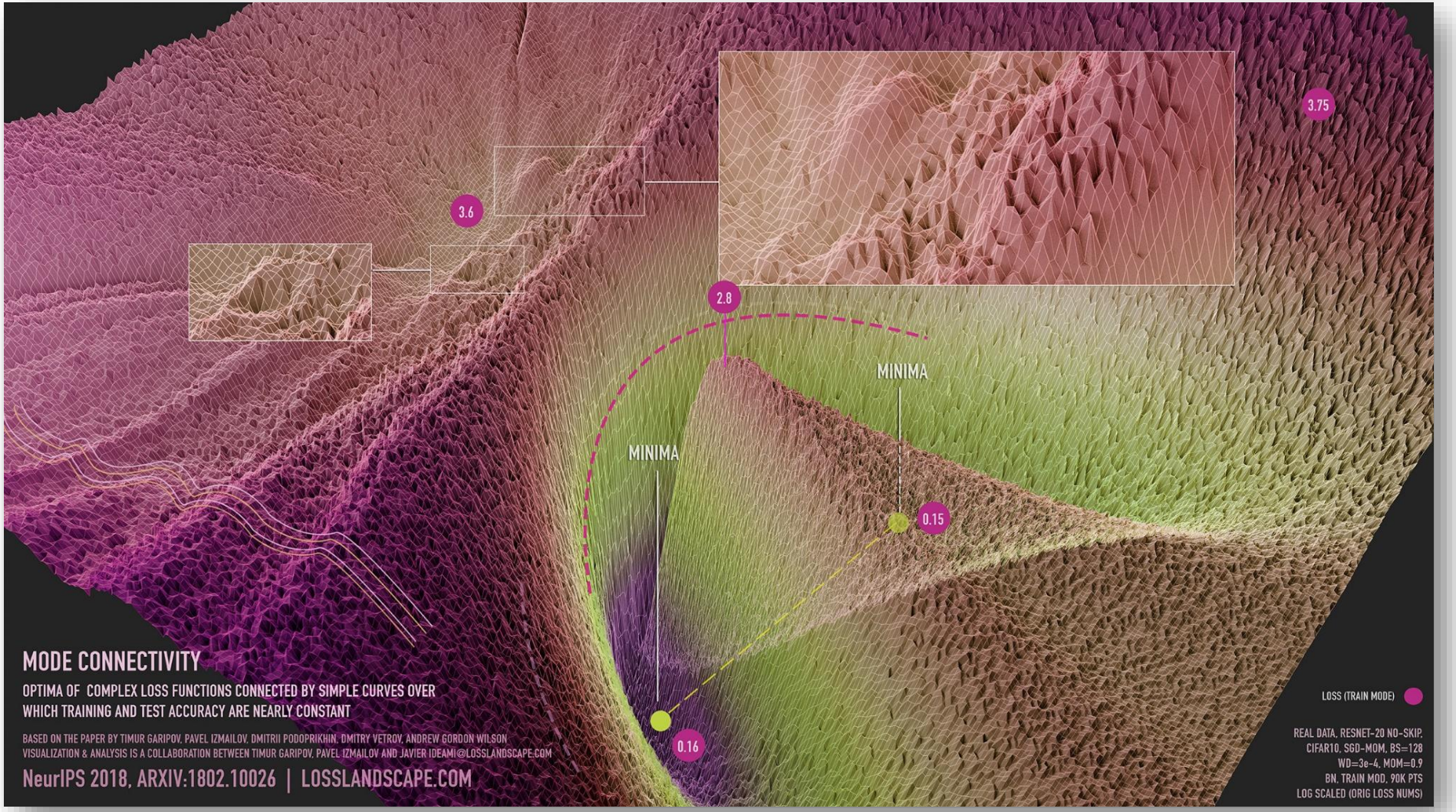


logistic regression



(small) neural network

The loss surface can be highly complex, with many local minima

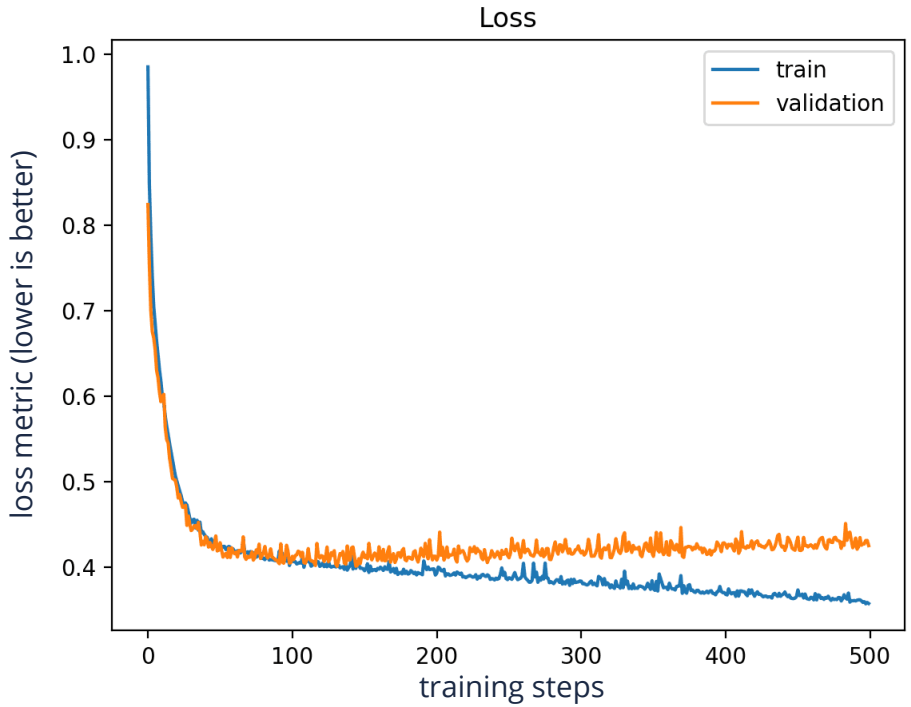


deep neural network

Neural networks are trained over multiple *epochs*

General training loop

- For each epoch
  - For each mini-batch (one step)
    - Forward pass
      make predictions
    - Calculate loss
      compare with targets
    - Calculate gradients (aka backward pass)
      how should parameters be updated?
    - Update model parameters
  - Forward pass on validation data
    - Did model improve? Continue.
    - Did it stop improving? Stop training.



When should you stop fitting this model?

Figure from https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/

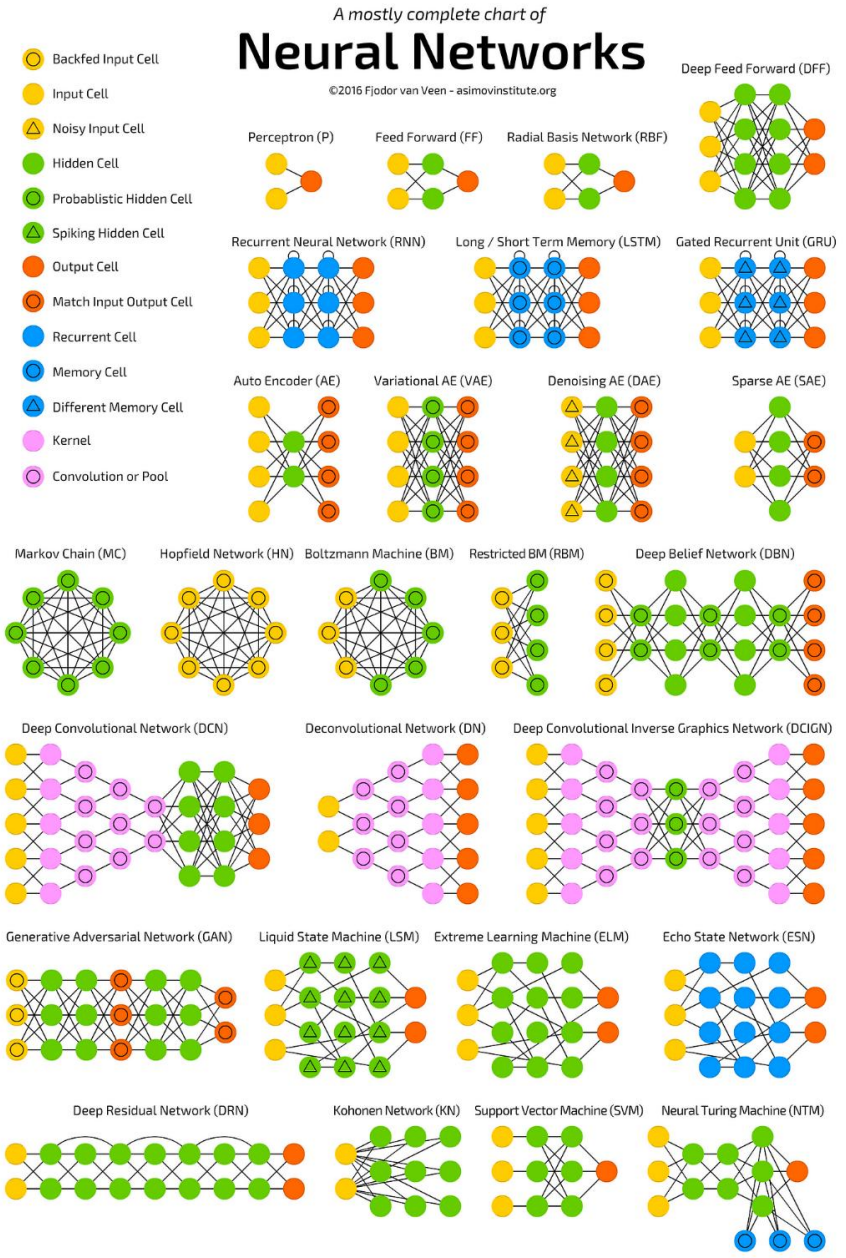
Neural networks are highly flexible

modules == building block == transformation == function

a module receives as input either data x or another module's output.

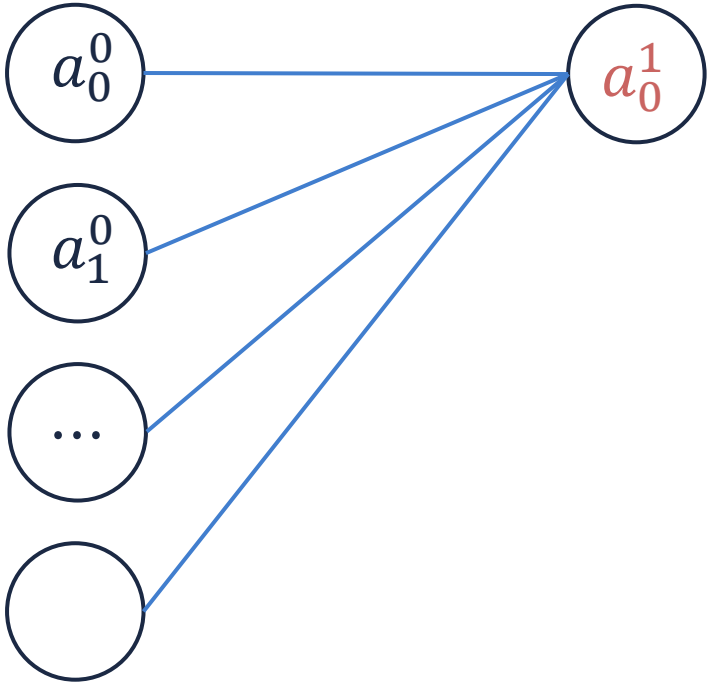
a module returns an output based on its activation function

a module may or may not have trainable model parameters.



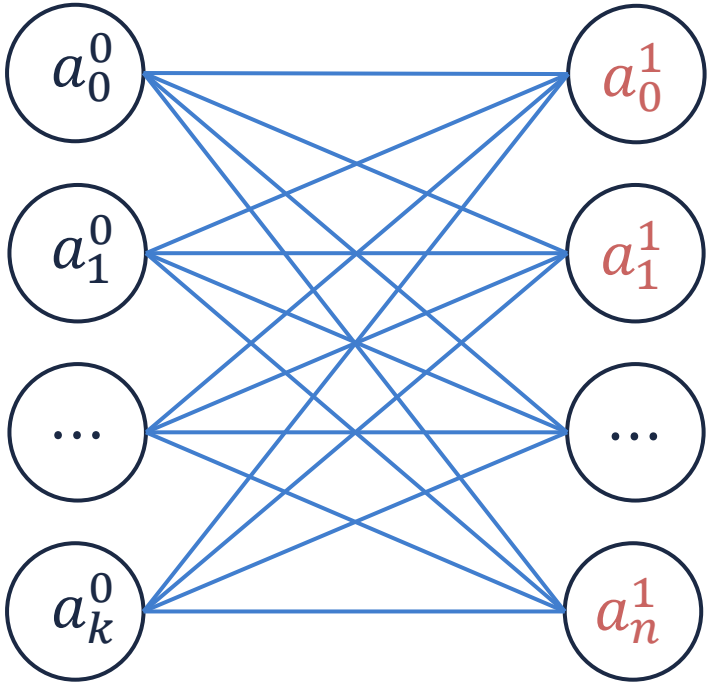
https://kim.hfg-karlsruhe.de/neural-network-chart/

A neural network layer can be represented as a matrix



$$a_0^1 = \sigma(w_{0,0} \cdot a_0^0 + w_{0,1} \cdot a_1^0 + \ ... + w_{0,n} \cdot a_n^0 + b_0)$$

A neural network layer can be represented as a matrix



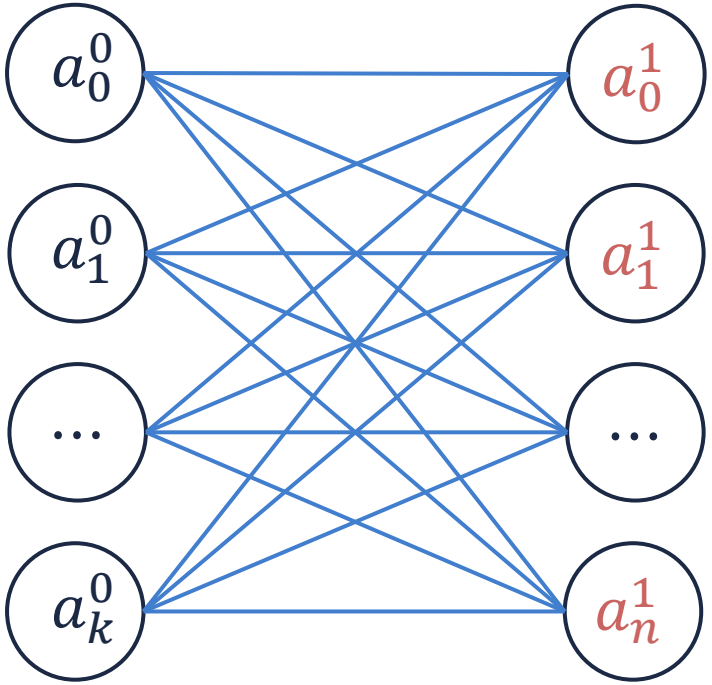
$$a_0^1 = \sigma(w_{0,0} \cdot a_0^0 + w_{0,1} \cdot a_1^0 + \ ... + w_{0,n} \cdot a_n^0 + b_0)$$

$$a_1^1 = \sigma(w_{1,0} \cdot a_0^0 + w_{1,1} \cdot a_1^0 + \ ... + w_{1,n} \cdot a_n^0 + b_1)$$

...

$$a_n^1 = \sigma(w_{k,0} \cdot a_0^0 + w_{k,1} \cdot a_1^0 + \ ... + w_{k,n} \cdot a_n^0 + b_n)$$

A neural network layer can be represented as a matrix



$$a_0^1 = \sigma(w_{0,0} \cdot a_0^0 + w_{0,1} \cdot a_1^0 + \ \ldots + w_{0,n} \cdot a_n^0 + b_0)$$

$$a_1^1 = \sigma(w_{1,0} \cdot a_0^0 + w_{1,1} \cdot a_1^0 + \ \ldots + w_{1,n} \cdot a_n^0 + b_1)$$

…

$$a_n^1 = \sigma(w_{k,0} \cdot a_0^0 + w_{k,1} \cdot a_1^0 + \ \ldots + w_{k,n} \cdot a_n^0 + b_n)$$

$$\begin{bmatrix} a_0^1 \\ a_1^1 \\ \vdots \\ a_n^1 \end{bmatrix} = \sigma\left(\begin{bmatrix} w_{0,0} & w_{0,1} & \cdots & w_{0,n} \\ w_{1,0} & w_{1,1} & \cdots & w_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{k,0} & w_{k,1} & \cdots & w_{k,n} \end{bmatrix} \begin{bmatrix} a_0^0 \\ a_1^0 \\ \vdots \\ a_n^0 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_n \end{bmatrix} \right)$$

$$a^1 = \sigma(W^1 \cdot a^0 + b^1)$$


GEFORCE GTX
GTX 970

Recap

- Neural networks can be understood as layered stacks of logistic-regression-like units.
- They can learn non-linear patterns through activation functions.
- Deeper networks can learn more complex patterns, but require more data.
- Backpropagation allows for gradient descent across many neural network layers.
- Mini-batch gradient descent balances speed and stability.
- Neural networks can be trained over many epochs.